DevOps: Project

Due on October 11, 2026

End-to-End DevOps Pipeline for a messaging web application

Nolan Blanc, Halan Ouensanga

Contents

Architecture and DevOps processes **3**
- Front-end . . . 3
- Back-end . . . 3
- Infrastructure and CI/CD Processes . . . 3
- Git hooks . . . 4

Design decisions **4**

Limitations and trade-offs **5**

Potential Future Work **5**

AI Usage **6**

This report provides a high-level overview of our project, presenting the different technologies and tools used to implement the application, the different components of the architecture and their interactions, before diving deeper into our design choices. Then, we highlight the design decisions we made regarding the different DevOps processes put in place, before discussing some of the trade-offs that we faced, and the overall limitations of our work.

Architecture and DevOps processes

Our project is a simple messaging web application, through which users can create an account, send messages, and manage their messaging groups. It was originally created as a lab assignement, as part of a Web development course at our home institution ENSIMAG, and we wanted to apply DevOps practices to it by building a CI/CD pipeline with GitHub Actions.

Our central idea was to transfer the exact same image that is used in the staging environment and carefully deploy it to production after the code has been tested in staging: we transition from one stable version to another.

Front-end

The web application uses React (jsx) for the front-end logic. The UI components are provided by shadcn.

When it comes to testing, linting to check coding style and quality is done using ESLint. Vitest[1] and React Testing Library help us define unit and integration tests on our React components to mock the back-end, while Cypress allows us to create multiple End-To-End (E2E) test scenarios to validate the application workflow, by simulating user interaction. Then, the entire app workflow is tested from the user's point of view, checking the that the UI and the features work as intended. We ensure that everything that can be seen or done by the user is tested through multiple use cases, like logging in as to test authentication, or creating groups to test specific features.

Back-end

The app uses a PostgreSQL database defined with Sequelize, an Object-Relational Mapping tool (ORM): this kind of tool makes it possible to use object notation to define our tables, our entities and the relational constraints between them. This concerns the "Model" part of our application. Sequelize also helps managing seeds and migrations, but it can be tricky on production environments and should be tested first on a disposable database. Then, classically, we define our REST API controllers with the Express.js framework. A websocket is used for real time messaging and few other features, to include have real time reactions; we use socket.io for websockets.

We can run and test back-end locally thanks to Node. The APIs tests are set up using Jest and Supertest: different calls to all of our API routes are defined, which then triggers different requests to the database. We designed only integrations tests to verify both our APIs and our database, and their interactions, checking the results and if errors are returned when they should be.

Infrastructure and CI/CD Processes

The development process for the application rests on three different environments:

- A local environment on our computer
- A staging environment corresponding to our main branch: it is designed to be as close as possible to our production environment, so that the tests we run matter
- A production environment hosting our main application, on a stable and validated version. We use a *build one, deploy many* politic with immutable docker images stored in our elastic container registry on AWS. This means that to deploy the next production release, we use a Docker image built for the staging environment that has been fully tested. The app deployed in production is bit-to-bit the same as the one tested in staging; we use the commit SHA to identify an image and used that same one again.

Our different environments are Docker instances running on AWS EC2 virtual server instances: we use Terraform to provision those virtual servers and their resources. Using Terraform, we define the infrastructure state of our servers, the resources (RDS database to go with our app, network rules configuration[2], OIDC configuration to give GitHub runner the permission to modify the state and communicate with instances) that they will require, and the roles and policies regulating access to those resources.

The processes are as follows:

[1] A compatile tool for testing applications built with Vite

[2] Such as ingress, egress, Virtual private cloud, ...

- On every pull request we ensure that the code is up to some quality (linting, plus basic unit tests) and security standards, and that it builds correctly.

  In particular, to check if our app contains any security vulnerabilities, npm-audit is ran on the front-end and back-end code to check for any dependencies or dependency version with security vulnerabilities (except on devDependencies, for which it raises and error on high or criticals vulnerabilities). Then our code is scanned for any hard coded secrets with the Gitleaks action.

- For pull requests on the develop branch, we additionally[3] run more complete integration tests: these are the whole set of sets that we defined for the front-end (Vitest, React Testing Library) and back-end (Jest and Supertest) during development.

- The main branch hosts the code that is tested and deployed on the staging environment. On a pull request we run an E2E job that uses the newly built image to test migrations (if there is one), and then run the Cypress tests in an environment that closely mirrors the AWS setup (without the network configuration or the RDS database). We also verify that our infrastructure definition follows conventions/rules with checkov, and at the same time generate the Terraform plan and post it as a comment on the PR, to give the reviewer an quick summary of the incoming changes before merge. This plan only reflects reality if no commits modifying the infrastructure are pushed to the main branch before this PR is merged.

- Each push on main triggers the deployment in the staging environment, applying the infrastructure staging state on AWS. We build the application image (or retrieve a previous build from the cache), and if both jobs passed we then send a deployment script to the EC2 instance through AWS Systems Manager (SSM). Yet, the deployment on staging is simple and unsafe for production releases.

- Once the team validates a version in our staging environment, the exact same image transfers to the production environment by tagging the corresponding commit. We tag the commit to release, and with a manually triggered workflow we call the Terraform *plan* and *apply jobs* on the production environment, and then promote the Docker image from staging to production (we get the commit SHA with the tag given as input). The deployment script is executed with SSM and we use a blue-green deployment approach to ensure a smooth transition for the user and facilitate rollbacks: we stop the current docker container and replace it we the new version, to have the option of rolling back if an error occurred. When the repository is public, the Terraform *apply* job needs a confirmation to execute: this is made possible thanks to the GitHub environment.

Dependabot is configured to open weekly pull requests on the dev branch for outdated or vulnerable npm, Docker, and GitHub Actions dependencies, so updates are reviewed in isolation before they reach staging or production.

Git hooks

Git hooks run locally before code is shared. On every commit, pre-commit runs lint-staged: Prettier and ESLint fix only the staged JavaScript files and terraform fmt format the staged Terraform files, so formatting and lint stay consistent without waiting for CI. On every push, pre-push runs the front-end unit tests and the full back-end Jest suite; if either fails, the push is blocked. That catches broken tests on the developer machine instead of after a pull request, keeps the shared branches cleaner, and reduces wasted CI runs.

Design decisions

About Terraform resources allocation, we decided to create a module app-stack that contains all the common infrastructure that can be customized with variables and we call this module in each environment with their own variables values. This helps removing code duplication and guaranties parity between staging and prod. Each environment has it own directory and we then use Terraform in one of those directory to target an environment. This solution is better than Terraform workspace because it allows more reusability of shared components, better possibility of customisation and offers better isolation between both environments (separate back-end state and credentials). A workspace stays interesting for PR environments, but it's not our objective here.

Thus, we created a "shared" folder holding the resources that all environments depend on and that the pipeline cannot provision for itself: the S3 bucket storing the Terraform state, the private ECR repository storing our Docker images, and the GitHub OIDC identity provider together with the IAM roles and policies that GitHub Actions assumes to manage AWS resources. Since the pipeline cannot authenticate before these roles exist, shared is applied manually from a local machine with the AWS CLI, using temporary admin credentials from a bootstrap

[3] We ensure that the code first goes through the basic security checks by preventing code from being merged, either if the pipeline fails or if it is not reviewed. In practice, setting up the repository to require a code review in order to merge review if the pipeline fails helps checking if the coding style that a company or a team adopts is correctly respected

Design decisions continued on next page...

user.

Since this is a school project running on AWS free-tier credits, with no availability requirements and a very light load on the application and the database, each persistent environment (staging and production) consists of a single EC2 instance and a single-AZ RDS instance. This keeps costs minimal and the infrastructure simple: we reuse the account's default VPC instead of designing our own network, and we need neither a load balancer nor a multi-AZ setup. As a consequence the current state lacks redundancy, network isolation relies on security groups rather than on private subnets, and both components are single points of failure: if the EC2 instance, the RDS instance, or the whole Availability Zone fails, the environment stays down until it is recreated from the Terraform code and the database is restored from backup. Blue-green deployment prevents downtime during production releases, but not infrastructure failures.

On another note, let us be reminded how our CI workflows re-use the tests that we defined when implementing our app, to check that our application worked correctly before powerful we pushed it. Even though designing tests as we code is a simple concept, this show how useful it is and how well it matches the DevOps mindset: by keeping a folder of the tests we design, and updating it with more tests when adding more features, we can use those same tests in our pipeline to check that our application works as intended on an different environment from our local one. Additionally, older tests may act as non-regression tests, checking that previously implemented features aren't don't break with new changes.

Limitations and trade-offs

The Git orchestration follows principles given in this post. Keeping a linear history and dealing with merge conflicts is hard when two branches ('main' and 'develop') stay active for a long period. We could improve rulesets and branch protection rules.

First, our test suites are limited: the coverage isn't what a real production app would expect. We refactored Cypress tests that mocked the back-end and database to have real tests that require all components of the architecture to be working in order to pass (real E2E). We then only wrote a few front-end tests to use in the CI, as our focus was more on CI design and deployment rather than increasing test coverage. A test on groups is flaky only when run on AWS instances, and we are still investigating to find the root cause (sockets are a lead, but we are not sure).

There is only a single role for GitHub Actions OIDC that has all the permissions necessary for the entire pipeline, with an overly broad allowed resource scope (many `Resource = "*"`). This should be split into a Terraform role with permissions to modify the state, and a deploy role with permissions to communicate with the instances, or even separate staging and production roles.

The pipelines do not use path filtering, which means even a documentation commit will trigger a new CI/CD process; this should be improved.

Caching is not as optimized as it could be, and we did not ensure that all workflows are hardened against attackers (we lack knowledge in this area).

Potential Future Work

Some important features could be added to our pipeline. For example, we could have made use of SonarQube, which is a well-known tool for improving the quality and security of code by analyzing code quality, measuring test coverage, and scanning for secrets. This was in the initial plan, but we chose to skip this step and focus on the rest of the pipeline: setting up a self-hosted SonarQube server should be done using a VPN for security to restrict access to it. Setting up a VPN would have taken too much time compared to the scope of this step, so time was allotted to the rest of the project instead, though this feature could be added with more time.

Using a VPN would have also allowed us to restrict access to the staging environment, which we did not do in our case. It is normally done through private networks or by whitelisting certain IP addresses.

Another missing feature is monitoring and observability. In real teams, and especially with applications with a large user base, the work doesn't stop after deploying code to production: an application has to be monitored continuously to observe how it performs and to check for anomalies. Future work on this app could include using Prometheus to monitor application performance or failures: for instance, by checking request response times or the rate of returned errors.

Potential Future Work continued on next page...

Finally, regarding observability, using the OpenTelemetry framework to normalize and centralize logs combined with AWS CloudWatch would have been a great addition. Such a tool saves teams precious time when trying to pinpoint the root cause of failures. Using it on a small-scale project like this one would have made for a great introduction.

AI Usage

Neither Claude Code nor Codex was used to code this project; we used Cursor and the Copilot free tier to help refactor the E2E tests. AWS has a lot of resources, and creating the policy permissions for OIDC can be tricky and took us a lot of time, so we used a bit of GenAI to help us produce that part. That is also why the permissions are a bit generous. We also used CloudTrail in AWS to understand what permissions were necessary, but it is still very time-consuming.

Then, AI was mainly used to address questions—about trade-offs we considered for the infrastructure design (like, for example, using Terraform workspaces to get the pros and cons faster than testing it or doing basic web research) or questions related to AWS resources to get more explanations, things like this. It was also used a bit for debugging Cypress (a nightmare) and misconfigurations, or to understand pipeline failures on GitHub with Copilot.